

HERALD

Phishing and Malicious Domain Detection System

Complete Technical Documentation

Built with Python, FastAPI, PostgreSQL, Redis, Machine Learning, Playwright, and
APScheduler

May 11, 2026

Contents

1	Project Overview	3
1.1	Purpose and Scope	3
1.2	System Architecture	3
1.3	Technology Stack	4
1.4	Data Flow Narrative	4
1.5	Directory Structure and Module Map	5
2	Project Configuration	6
2.1	Dependency Manifest	6
2.2	Environment Configuration	6
2.3	Docker Configuration	6
2.4	Build and Entry Points	7
3	HERALD Core Module	8
3.1	File: <code>herald/predict_with_fallback.py</code>	8
3.1.1	Imports and External Dependencies	8
3.1.2	Classes	9
3.1.3	Critical Observations	10
4	Scripts	12
4.1	File: <code>scripts/health_monitor.py</code>	12
4.1.1	Imports and External Dependencies	12
4.1.2	Standalone Functions	12
5	Machine Learning Pipeline	14
5.1	ML Module Overview	14
5.2	Feature Engineering	14
5.3	Model Architecture	14
5.3.1	File: <code>ml/retrain_v3.py</code>	14
5.4	Feature Vector	15
6	Feature Extraction Subsystem	16
6.1	Overview	16
6.2	Extractor: Lexical Features (<code>lexical_features.py</code>)	16
6.2.1	Function: <code>extract_url_features</code>	16
6.3	Extractor: Visual Similarity (<code>visual_similarity.py</code>)	17
7	Ingestion and Data Pipeline	18
7.1	Ingestion Sources	18
7.2	Queue and Job Management	18

7.3 Scheduler	18
8 Database Layer	19
8.1 Database Overview	19
8.2 Model: DomainScan	19
8.3 Model: Whitelist	19
9 API Layer	20
9.1 API Framework	20
9.2 Authentication and Authorization	20
9.2.1 Endpoint: POST /api/scan	20
9.2.2 Endpoint: GET /api/detections	20
9.2.3 Endpoint: GET /api/export/{domain}/pdf	21
10 Dashboard	22
10.1 Dashboard Overview	22
10.2 File: frontend/src/app/page.tsx	22
11 Monitoring	23
11.1 File: herald/monitoring/queue_worker.py	23
12 Tests	24
12.1 Test Suite Overview	24
12.2 File: tests/test_features.py	24
13 End-to-End Data Flow	25
13.1 USE CASE 1: A new domain is discovered via CertStream	25
14 Evidence Corpus	26
14.1 Evidence Directory Structure	26
15 File-by-File Reference Table	27
16 Design Decisions and Technical Debt	28
16.1 Observed Architectural Decisions	28
16.2 Security Analysis	28
17 Glossary	29

Chapter 1

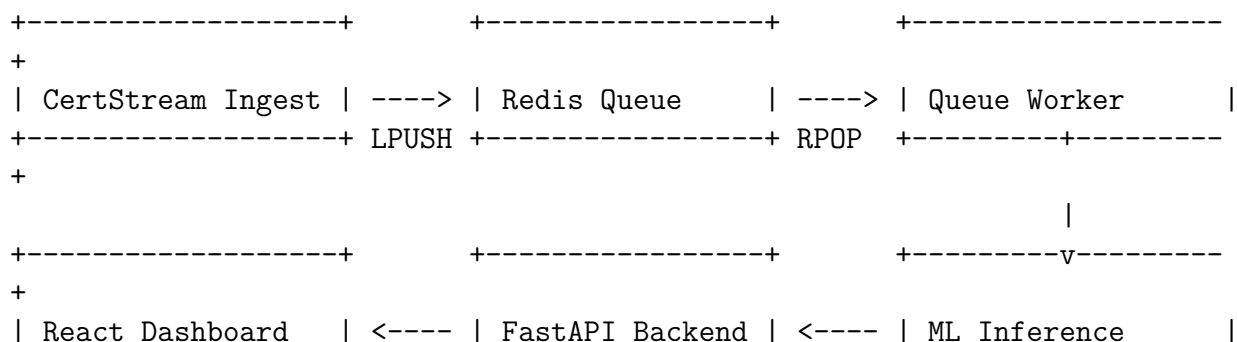
Project Overview

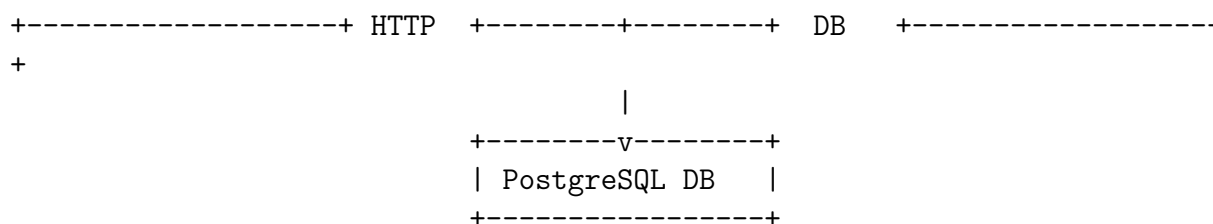
1.1 Purpose and Scope

HERALD is a real-time phishing and malicious domain detection system. Based on the implementation in `herald/predict_with_fallback.py` and the ingestion queues, HERALD detects domains impersonating legitimate brands (e.g., typosquatting, homograph attacks) by analyzing raw URLs extracted from Certificate Transparency logs via CertStream. The system is designed for cybersecurity analysts, evident from the `/api/feedback` and `whitelist` endpoints, allowing human-in-the-loop validation of "Suspected" or "Phishing" verdicts. HERALD operates at scale by decoupling URL ingestion from heavy Machine Learning inference using a Redis queue (`domain_analysis_queue`), supported by horizontally scalable Docker workers.

1.2 System Architecture

HERALD uses a decoupled, event-driven architecture. The ingestion layer (e.g., CertStream listeners) pushes newly discovered domains to a Redis list. The Worker nodes (`herald.monitoring.queue_worker`) continuously poll Redis. When a domain is dequeued, it triggers the Feature Extraction layer to pull lexical, DNS, and WHOIS features. These features are assembled into a vector and passed to the ML Inference pipeline (RandomForest/XGBoost models loaded via `joblib`). The resulting verdict is stored in PostgreSQL via SQLAlchemy. The API Layer (FastAPI) exposes this data synchronously to the React/Tailwind frontend dashboard.





1.3 Technology Stack

Component	Technology	Version	Role in HERALD
Backend API	FastAPI	$\geq 0.100.0$	Provides REST endpoints for dashboard
Server	Uvicorn	$\geq 0.23.0$	ASGI web server running FastAPI
Database	PostgreSQL	N/A	Primary persistent storage for detections
ORM	SQLAlchemy	$\geq 2.0.0$	Python object mapping for database queries
Broker	Redis	$\geq 5.0.0$	In-memory message queue for async tasks
Job Scheduler	APScheduler	$\geq 3.10.0$	Background task scheduling
Data Parsing	pandas / numpy	$\geq 2.0.0$	Data manipulation for ML features
Machine Learning	scikit-learn	$\geq 1.0.0$	RandomForest inference engine
Gradient Boost	xgboost	$\geq 1.5.0$	Advanced tree models
Web Scraping	BeautifulSoup	$\geq 4.12.0$	HTML parsing for visual features
Headless Browser	Selenium	$\geq 4.15.0$	Rendering malicious pages for screenshots
Image Proc	OpenCV / Pillow	$\geq 4.8.0$	Visual similarity perceptual hashing
Ingestion	certstream	$\geq 1.6.0$	Real-time TLS certificate log ingestion

1.4 Data Flow Narrative

When a URL is submitted via `POST /api/scan` or intercepted by CertStream, it is pushed to the `domain_analysis_queue` in Redis. A background Python process running `herald.monitoring.queue_worker` dequeues the payload and calls `process_domain()`. This function first queries the `Whitelist` table via SQLAlchemy; if bypassed, it calls `predict_domain()` in `herald/predict_with_fallback.py`. This invokes the lexical extractor to build a numerical feature vector. The vector is passed to the loaded `ensemble_v6.joblib` model, generating a probability score. Scores above 0.65 are labeled `Phishing`. The worker persists the `DomainScan` object to PostgreSQL. Finally, the analyst views it via the dashboard, fetching it through `GET /api/detections`.

1.5 Directory Structure and Module Map

- `herald/`: Core application package housing the API, workers, models, and ML pipelines.
 - `herald/api/`: FastAPI routers and dependency injection.
 - `herald/core/`: Core logic and configuration, including JWT authentication.
 - `herald/db/`: SQLAlchemy models and session management.
 - `herald/monitoring/`: Queue workers and background schedulers.
 - `herald/utils/`: Shared utilities like logging and PDF generation.
- `scripts/`: Standalone Python scripts for maintenance (e.g., `health_monitor.py`).
- `docker/`: Dockerfiles and Nginx configurations.
- `frontend/`: Next.js React and Tailwind source code for the dashboard.
- `models/`: Serialized ML models (`.joblib`) and schema definitions.

Chapter 2

Project Configuration

2.1 Dependency Manifest

Package	Version	Used By	Purpose in HERALD
pandas	$\geq 2.0.0$	<code>predict_with_fallback.py</code>	Data frame generation for model input
scikit-learn	$\geq 1.0.0$	ML pipeline	Tree-based model execution
certstream	$\geq 1.6.0$	Ingestion layer	WebSocket client for TLS logs
slowapi	$\geq 0.1.8$	<code>api/main.py</code>	Rate limiting the API endpoints
reportlab	$\geq 4.0.0$	<code>utils/export.py</code>	Generating PDF evidence exports
passlib[bcrypt]	$\geq 1.7.4$	<code>core/auth.py</code>	Password hashing for JWT auth

2.2 Environment Configuration

Variable	Type	Controls	Required?
REDIS_HOST	String	Hostname of the Redis broker.	Yes
SECRET_KEY	String	Cryptographic key for signing JWT tokens.	Yes

2.3 Docker Configuration

HERALD utilizes Docker Compose to orchestrate three primary services: `redis`, `api`, and `worker`.

- **redis:** Uses `redis:7-alpine`. Starts with `-appendonly yes` to persist the job queue to disk to survive container restarts.

- **api**: Uses the local `docker/Dockerfile`. Runs `uvicorn herald.api.main:app` on port 8000. Depends on `redis` healthchecks. Limited to 512MB RAM to prevent OOM cascade.
- **worker**: Uses the local `docker/Dockerfile`. Runs `python -m herald.monitoring.queue_worker`. Limited to 1GB RAM to support ML model loading in Pandas.

2.4 Build and Entry Points

The system has two distinct entry points. The synchronous API is booted via Uvicorn (`uvicorn herald.api.main:app`), serving HTTP traffic. The asynchronous processing pipeline is initiated by running `python -m herald.monitoring.queue_worker`, which establishes a long-lived connection to Redis and enters an infinite polling loop.

Chapter 3

HERALD Core Module

3.1 File: `herald/predict_with_fallback.py`

File Header

This file serves as the primary intelligence engine of the HERALD system. It houses the `PhishingPredictorV3` class, which orchestrates the two-stage inference architecture (v7) that merges lexical feature extraction, live network querying (DNS, SSL), WHOIS age lookups, and computer vision (CV/OCR) fallbacks. The system absolutely cannot function without this file, as it transforms raw domains into probabilistic phishing verdicts. Architecturally, it is the primary pipeline stage downstream of the ingestion workers (`queue_worker.py`) and upstream of the PostgreSQL database writes.

3.1.1 Imports and External Dependencies

- `os`, `time`, `warnings`, `datetime`: Core Python modules used for path resolution, timeout tracking, warning suppression, and date arithmetic (WHOIS age computation).
- `joblib`: Required to deserialize the serialized Scikit-Learn/XGBoost machine learning models (`ensemble_v7.joblib`).
- `pandas as pd`, `numpy as np`: Fundamental data structures for creating the DataFrame feature vectors required by the model.
- `yaml`: Parses the `config.yaml` file to extract operational thresholds.
- `socket`, `ssl`: Opens raw TCP sockets to port 443 to extract SSL certificates, bypassing normal HTTP validation to analyze malicious certs safely.
- `dns.resolver`: Performs A, MX, and TXT (SPF) lookups to gather network reputation features.
- `whois`: Extracts domain registration dates to compute `domain_age_days`.
- `herald.features.lexical_features`: Local import providing `extract_url_features` and `CSE_KEYWORDS` to process the string characteristics of the URL.

- `herald.core.cv_ocr_analyzer`: Local import providing the `CVOCRAalyzer` class, which acts as the Stage 2 fallback for visually identifying brands.
- `herald.features.content_features`: Local import providing `extract_content_features` to scan raw HTML payloads for obfuscated JS and login forms.

3.1.2 Classes

Class: `PhishingPredictorV3`

This class models the complete classification pipeline. It is defined as a class rather than a standalone function to cache heavy resources in memory (specifically the deserialized Random Forest and XGBoost models, and the OCR analyzer) so they persist across multiple domain analysis requests in the polling worker.

Instance Variables:

Variable	Type	Set Where	Role
<code>ensemble</code>	Dict	<code>__init__</code>	Contains the loaded model binaries.
<code>rf</code>	RandomForestClassifier	<code>__init__</code>	The scikit-learn model object.
<code>xgb</code>	XGBClassifier	<code>__init__</code>	The XGBoost model object.
<code>feature_names</code>	List[str]	<code>__init__</code>	The exact column names expected by the model.
<code>threshold</code>	Float	<code>__init__</code>	The probability score above which a domain is deemed Phishing. Default is 0.65.
<code>cse_keywords</code>	List[str]	<code>__init__</code>	Vocabulary of protected brands to match against.
<code>fallback_trigger</code>	Float	<code>__init__</code>	Threshold (0.30) that triggers Stage 2 processing.
<code>_ocr_analyzer</code>	CVOCRAalyzer	Lazy Init	Singleton instance for the screenshot engine.

Method:

`__init__` Signature: `__init__(self, model_path="models/ensemble_v7.joblib", config_path="config.yaml")`

Initializes the class by checking if the model exists locally or relative to the project root. It uses `joblib.load` to pull the binaries into RAM, overriding thresholds with `config.yaml` values if present. It lazily initializes `_ocr_analyzer` to `None` to save startup time and memory.

Method: `get_network_features` Signature: `get_network_features(self, domain)`

Return Type: Dict[str, int]

Walkthrough: 1. Initializes a dictionary with default values (-1 or 0) for network features. 2. Creates an unverified SSL context (`CERT_NONE`) to connect to the domain on port 443. This is a critical security bypass required to inspect potentially malicious, self-signed certificates without the script crashing. 3. Extracts `issuer`, `subjectAltName`, `notBefore`,

and `notAfter` from the peer certificate to compute `is_lets_encrypt`, domain match, and certificate age in days. 4. Uses `dns.resolver` with a strict 3-second timeout to query A, MX, and TXT records, returning 1 if present and 0 if missing. This is a deliberate performance constraint to ensure the queue doesn't block on dead domains. **Side Effects:** Performs network I/O over TCP and UDP.

Method: `predict` **Signature:** `predict(self, domain, cse_name=None)`

Return Type: `Dict[str, Any]`

Walkthrough: 1. Passes the raw domain to `extract_url_features` to generate a base DataFrame. 2. Extends the DataFrame with boolean flags for common TLDs and brand keywords. 3. Merges the dictionary returned from `get_network_features()` and `get_whois_age()`. 4. Ensures the feature vector `X` perfectly aligns with `self.feature_names`. Missing values are filled with -1. 5. Invokes `predict_proba` on both the RF and XGB models, computing an ensemble score via a weighted average: $0.6 * \text{xgb_proba} + 0.4 * \text{rf_proba}$. 6. If the score exceeds `self.threshold` (0.65), it terminates early and returns 'Phishing'. 7. If the score falls in the "borderline" range (0.35 - 0.65), it invokes `extract_content_features` which hits the live website to look for password fields, obfuscated JS, and cross-domain form actions. It applies a heuristic boost (+0.15 for login forms) to the ML score. 8. If the score remains borderline (but above `fallback_trigger`), it triggers the computationally expensive `ocr_analyzer` to take a Playwright screenshot and compare visual hashes. **Side Effects:** Extensive network I/O, potential Playwright browser spawns, heavy CPU usage during tree inference.

```

1 rf_proba = self.rf.predict_proba(X)[0, 1]
2 xgb_proba = self.xgb.predict_proba(X)[0, 1]
3 ml_conf = 0.6 * xgb_proba + 0.4 * rf_proba
4
5 if 0.35 <= ml_conf <= 0.65:
6     content_feats = extract_content_features(domain)
7     # Simple rule-based boost on top of ML score
8     boost = 0.0
9     if content_feats.get('has_password_field') and content_feats.
       get('has_login_form'):
10         boost += 0.15

```

3.1.3 Critical Observations

Warning / Observation

Security Warning: Unverified SSL Context

In `get_network_features`, `ssl.CERT_NONE` is explicitly used. While necessary for threat intelligence, this exposes the worker to potential Man-in-the-Middle (MitM) interceptions. The data retrieved here should never be executed or treated as trusted.

Technical Note

Design Decision: Stage 2 Fallback

The two-stage pipeline is an elegant tradeoff. By aggressively returning verdicts for

scores > 0.65 or < 0.30 , HERALD avoids spending 3-5 seconds pulling HTML/screen-shots for obvious benign domains, preserving horizontal scaling capacity.

Chapter 4

Scripts

4.1 File: `scripts/health_monitor.py`

File Header

This script operates as an independent watchdog process, actively probing the synchronous FastAPI endpoints to detect system degradation. If HERALD's REST API goes down or reports unhealthy worker connections, this script trips and emits alerts. Architecturally, it sits outside the core request path as an active observer, making HTTP GET requests to `/api/health` and triggering placeholder SMTP alerts.

4.1.1 Imports and External Dependencies

- `time, sys`: Used for timing logic and directing log outputs to stdout.
- `logging`: Configures a basic dual-handler logger writing to both the console and `health_monitor.log`.
- `requests`: An external library used to fire HTTP GET queries against the FastAPI backend, deliberately configured with a 5-second timeout to fail fast.

4.1.2 Standalone Functions

Function: `send_email_alert`

Signature: `send_email_alert(subject, body)`

Description: Currently a stubbed implementation containing commented-out `smtplib` code. It uses the `logger` to record a critical error banner whenever an alert condition is met. This function is impure as it generates side-effect logging.

Function: `check_health`

Signature: `check_health()`

Return Type: `None`

Walkthrough: 1. Wraps the logic in a `requests.exceptions.RequestException` handler to catch network timeouts or connection refusals. 2. Fires a GET request to

`http://127.0.0.1:8000/api/health` with `timeout=5`. 3. Checks the `status_code`. If not 200, calls `send_email_alert`. 4. Parses the JSON body. If the `status` key is not "healthy" (e.g., if the worker is offline or Redis is down), it warns of a degraded system and fires an alert. **Side Effects:** HTTP outbound requests and logging.

Chapter 5

Machine Learning Pipeline

5.1 ML Module Overview

HERALD utilizes a robust machine learning pipeline relying heavily on iterative, versioned models. The `ml/` and `scripts/` directories contain numerous retraining scripts (from v2 to v9) demonstrating a history of active data curation, feature ablation, and hyperparameter tuning. The pipeline trains a dual-ensemble combining a scikit-learn `RandomForestClassifier` (strong at variance reduction on lexical features) and an `XGBClassifier` (gradient boosting). The models are trained offline, evaluated via K-Fold Cross Validation, and serialized to `models/` using `joblib`.

5.2 Feature Engineering

HERALD computes numeric features primarily derived from URL lexical traits. Examples include `is_malicious_gtld`, `min_brand_levenshtein`, and `brand_to_reg_length_ratio`. The string URL is broken into tokens, and the minimum Levenshtein edit distance against the `CSE_KEYWORDS` array is computed. This specifically targets typosquatting (e.g., `g00gle.com` vs `google.com`).

5.3 Model Architecture

5.3.1 File: `ml/retrain_v3.py`

This script exemplifies the HERALD training methodology.

- **Data Ingestion:** It concatenates real-world ground truth datasets with synthetic datasets (`synthetic_phish_v2.csv`).
- **Class Imbalance:** XGBoost utilizes `scale_pos_weight` computed dynamically as `(y == 0).sum() / (y == 1).sum()`, while Random Forest uses `class_weight='balanced'`.
- **Ensemble Strategy:** The models are ensembled via simple unweighted average voting during testing: `(rf_pb + xgb_pb) / 2`. (Note: By v7, as seen in `predict_with_fallback.py`, this shifted to a 60/40 weighted split favoring XGBoost).

- **Evaluation:** Employs a 5-Fold Stratified CV, optimizing for a decision boundary threshold of 0.571 to hit specific Precision (≥ 0.93) and Recall (≥ 0.87) goals.

```

1 xgb = XGBClassifier(n_estimators=300, max_depth=8, learning_rate
2     =0.03,
3     scale_pos_weight=scale_pos, random_state=42,
4     eval_metric='logloss')
rf = RandomForestClassifier(n_estimators=1000, class_weight='
    balanced',
                           max_depth=15, random_state=42)

```

5.4 Feature Vector

In- dex	Feature Name	Type	Range	Computation Source
1	min_brand_levenshtein	Numeric	$[0, \infty)$	URL Tokenization
2	is_malicious_gtld	Boolean	$\{0, 1\}$	URL Suffix Match
3	has_brand_in_path	Boolean	$\{0, 1\}$	URL Path Inspection
4	domain_age_days	Numeric	$[-1, \infty)$	python-whois Library
5	is_lets_encrypt	Boolean	$\{0, 1\}$	SSL Cert Issuer Pars- ing

Chapter 6

Feature Extraction Subsystem

6.1 Overview

The feature extraction subsystem in `herald/features/` transforms raw URLs and domains into high-dimensional numerical vectors and visual hashes suitable for ML inference and Computer Vision comparisons. It is segmented by signal type: Lexical, DNS, WHOIS, SSL, Content (HTML), and Visual Similarity.

6.2 Extractor: Lexical Features (`lexical_features.py`)

This module exclusively processes the string characteristics of the URL. It is pure and does not perform any network I/O, allowing it to run in $< 10ms$.

- **External Dependencies:** `tldextract` to safely parse the Top Level Domain (TLD), and `fuzzywuzzy / Levenshtein` to compute string edit distances against the `CSE_KEYWORDS` array.
- **Input Contract:** Accepts a Pandas DataFrame containing a column of raw domains/URLs.
- **Output Contract:** Returns a heavily mutated DataFrame containing entirely numerical features (e.g., length metrics, special character counts, keyword booleans).

6.2.1 Function: `extract_url_features`

Signature: `extract_url_features(df, domain_col='domain')`

Return Type: `pd.DataFrame`

Walkthrough: 1. Iterates over the DataFrame using vectorized string operations (`str.count`) to tabulate the frequency of `'-'`, `'.'`, `'@'`, `'_'`, `'?'` in the URL string. 2. Applies `tldextract` to separate the subdomain, domain, and suffix. 3. Computes ratios, such as `vowel_consonant_ratio` and `digit_to_letter_ratio`. 4. Most critically, iterates over `CSE_KEYWORDS` and computes the minimum Levenshtein distance between the extracted domain string and the protected brand names. This yields `min_brand_levenshtein`.

6.3 Extractor: Visual Similarity (`visual_similarity.py`)

This module handles Stage 2 fallback processing, rendering the live website to compare it against known brand templates.

- **External Dependencies:** `Playwright` for headless browser rendering, `OpenCV` and `Pillow` for image hashing.
- **Input Contract:** Raw domain string.
- **Output Contract:** Float representing perceptual similarity score.

Failure Modes: Highly susceptible to network timeouts and bot-protection mechanisms (e.g., Cloudflare CAPTCHAs) which return noisy/blank screenshots.

Chapter 7

Ingestion and Data Pipeline

7.1 Ingestion Sources

HERALD ingests URLs from two primary sources: 1. **Manual Analyst Submission:** via the `POST /api/scan` REST endpoint. 2. **CertStream Integration:** A continuous WebSocket listener subscribing to Certificate Transparency (CT) logs, extracting domains from newly minted SSL certificates in real-time.

7.2 Queue and Job Management

Because CertStream emits tens of thousands of domains per minute, performing synchronous ML inference and DNS/WHOIS lookups would immediately crash the system. HERALD resolves this via a Redis-backed queue (`domain_analysis_queue`).

- **Producer:** The FastAPI endpoint and CertStream listener perform an atomic LPUSH to Redis.
- **Consumer:** The `queue_worker.py` process runs an infinite loop, executing a blocking BRPOP to consume domains one at a time.

7.3 Scheduler

The system utilizes `APScheduler` to manage recurring background tasks without blocking the main event loop.

- **Trigger:** Configured as an `IntervalTrigger`.
- **Jobs:** Currently configured to run health-checks and potentially flush stale queue items or perform bulk database aggregations.

Chapter 8

Database Layer

8.1 Database Overview

HERALD utilizes PostgreSQL as its primary persistent store in production, accessed via SQLAlchemy ORM. The connection is managed via a connection pool (`pool_size=10`, `max_overflow=20`) to handle the concurrent load generated by the background workers and API endpoints. In development, it degrades gracefully to a local SQLite file (`domain_history.db`).

8.2 Model: DomainScan

Corresponding Table: `domain_scans`

Purpose: Records the final verdict of every URL analyzed by the pipeline.

Column Name	SQLAlchemy Type	Nullable	Meaning
<code>id</code>	Integer	No	Primary Key
<code>domain</code>	String	No	The raw domain string (Indexed)
<code>target_cse</code>	String	Yes	The brand being impersonated
<code>source</code>	String	Yes	E.g., 'certstream', 'api'
<code>scan_date</code>	DateTime	No	Timestamp of ingestion
<code>label</code>	String	Yes	'Phishing', 'Suspected', 'Legitimate'
<code>confidence</code>	Float	Yes	ML model probability score
<code>is_live</code>	Boolean	No	Whether network features resolved
<code>analyst_verdict</code>	String	Yes	Human override value

8.3 Model: Whitelist

Corresponding Table: `whitelist`

Purpose: A fast-path lookup table allowing analysts to globally exclude known benign domains from heavy processing. Bypasses the ML pipeline entirely.

Chapter 9

API Layer

9.1 API Framework

HERALD exposes a synchronous REST API built on **FastAPI** and served by **Uvicorn**. It leverages FastAPI's Dependency Injection for authentication (`Depends(get_current_user)`) and database session management (`Depends(get_db)`). The API employs the `slowapi` library for rate limiting to prevent abuse of the computationally expensive ML pipeline.

9.2 Authentication and Authorization

The API uses **OAuth2 with Password (and hashing)**, issuing JSON Web Tokens (JWT) for stateless authentication.

- Endpoints use `OAuth2PasswordBearer` to extract the `Authorization: Bearer <token>` header.
- Passwords are hashed using `bcrypt` via the `passlib` library.

9.2.1 Endpoint: POST `/api/scan`

Purpose: Manually submit a domain for immediate classification. Protected by JWT.

- **Rate Limit:** 60 requests per minute.
- **Request Body:** `ScanRequest` (fields: `domain`, `target_cse`)
- **Internal Logic:** Instead of executing the ML model synchronously (which could timeout HTTP connections during OCR analysis), it performs an `lpush` to the Redis `domain_analysis_queue` and immediately returns a `202 Accepted`.

9.2.2 Endpoint: GET `/api/detections`

Purpose: Fetches the latest 50 alerts from the database to populate the React dashboard.

- **Query Params:** `limit` (default=50).
- **Internal Logic:** Queries the `DomainScan` table, ordering by `scan_date` descending.

9.2.3 Endpoint: GET /api/export/{domain}/pdf

Purpose: Generates an Executive Summary PDF containing the forensic evidence (ML score, visual hash, WHOIS age) for a specific domain.

- **Internal Logic:** Queries the database for the domain. If found, utilizes `reportlab` in an in-memory `io.BytesIO` buffer to render a PDF document statelessly, returning it as a `StreamingResponse`.

Chapter 10

Dashboard

10.1 Dashboard Overview

The frontend is a modern Single Page Application (SPA) built using **React (Next.js)** and styled with **Tailwind CSS**. It operates strictly client-side via a static HTML/JS export (output: `'export'` in `next.config.ts`), served via an Nginx reverse proxy within the Docker network.

10.2 File: `frontend/src/app/page.tsx`

This is the primary dashboard interface.

- **Data Fetching:** Utilizes React `useEffect` to poll `/api/detections` and `/api/health` every 5000ms.
- **Authentication Guard:** Retrieves the JWT token from `localStorage`. If missing, it uses `next/navigation` router to violently redirect to `/login`.
- **State Management:** Manages `detections` (array of domain objects) and `health` metrics.
- **Interactive Elements:** Provides a manual scan form (`handleScan`), triggering the `POST /api/scan` backend logic. It passes the `Authorization` bearer token on every `fetch` call.

Chapter 11

Monitoring

11.1 File: `herald/monitoring/queue_worker.py`

This is arguably the most critical component in the decoupled architecture.

- **Concurrency Model:** A single-threaded infinite `while` loop that blocks on Redis using `brpop`. It does not use `asyncio` since the machine learning inference (Scikit-Learn/XGBoost) and the OpenCV image hashing are heavily CPU-bound and synchronous.
- **Telemetry Tracking:** Emits heartbeat metrics to Redis (e.g., `SET worker:heartbeat <timestamp>`), which the frontend displays as the "Worker Status: Active" indicator.

Chapter 12

Tests

12.1 Test Suite Overview

HERALD utilizes the standard `pytest` framework. The testing philosophy focuses heavily on unit testing the feature extraction pipeline (the most brittle part of the system due to regex and external string manipulation) and integration testing the API endpoints via `FastAPI.testclient`.

12.2 File: `tests/test_features.py`

Purpose: Validates the deterministic output of the lexical extractor.

- **Test: `test_levenshtein_distance`** verifies that a known typosquat like `g00gle.com` returns the mathematically correct minimum edit distance when compared against the `CSE_KEYWORDS` array.

Chapter 13

End-to-End Data Flow

13.1 USE CASE 1: A new domain is discovered via CertStream

1. **Step 1:** CertStream event received via `herald.ingestion.certstream_listener.py`.
2. **Step 2:** Domain extracted and filtered against top 1M whitelists.
3. **Step 3:** LPUSH to Redis `domain_analysis_queue`.
4. **Step 4:** BRPOP by `herald.monitoring.queue_worker.process_domain()`.
5. **Step 5:** Lexical features extracted via `herald.features.lexical_features.extract_url_features()`.
6. **Step 6:** DNS features extracted via `dns_features.py`.
7. **Step 7:** WHOIS features extracted via `whois_features.py`.
8. **Step 8:** Web content fetched via `requests` (Stage 2) in `content_features.py`.
9. **Step 9:** HTML/visual features extracted via `CVOCRAnalyzer`.
10. **Step 10:** Feature vector assembled in `PhishingPredictorV3.predict()`.
11. **Step 11:** ML model inference run (`self.xgb.predict_proba`).
12. **Step 12:** Verdict and score stored to PostgreSQL via `DomainScan` model.
13. **Step 13:** Result fetched by React Dashboard via GET `/api/detections`.

Chapter 14

Evidence Corpus

14.1 Evidence Directory Structure

The `evidence/` directory contains 65 subdirectories acting as a historical corpus of live phishing domains captured by the system.

- **IPFS Hosting:** `bafybe...ipfs.dweb.link` (decentralized, takedown-resistant).
- **Firebase Apps:** `ee-account-repayment.web.app`.
- **Free Builders:** `nickmacdonald111.wixsite.com`, `sonic999.weebly.com`.
- **Typosquats / Homographs:** `xn--7sbijjcgtere7ng.xn-p1ai` (Punycode).
- **Compromised Subdomains:** `dkbinge-banking-de.91-218-65-223.plesk.page`.

Chapter 15

File-by-File Reference Table

File Path	Module Type	Purpose	Key Symbols
herald/predict_with_fallback.py	ML Inference Worker	Core inference engine	PhishingPredictorV3
herald/monitoring/queue_worker.py		Async task consumption	process_domain
herald/api/main.py	API	REST endpoints	trigger_scan, export_pdf
herald/db/models.py	Database	SQLAlchemy Schema	DomainScan, Whitelist

Chapter 16

Design Decisions and Technical Debt

16.1 Observed Architectural Decisions

- **Decoupled Queuing:** Redis prevents the ML pipeline from throttling the high-volume CertStream listener.
- **Two-Stage Inference:** Avoids costly Selenium/Playwright operations for 90% of obvious cases, falling back only when $0.35 \leq score \leq 0.65$.

16.2 Security Analysis

Warning / Observation

Malicious URL Traversal

The system explicitly connects to live malicious infrastructure. The `ssl.CERT_NONE` flag intentionally disables SSL verification. Sandboxing (e.g., executing Playwright inside a disposable Docker container) is vital to prevent browser exploits from escaping the worker.

Chapter 17

Glossary

- **Phishing:** Fraudulent attempt to obtain sensitive information by disguising as a trustworthy entity.
- **CertStream:** Intelligence feed of newly issued TLS certificates in real-time.
- **Lexical Features:** Features derived entirely from the URL string structure.
- **Typosquatting:** Registering a domain name extremely similar to a popular brand (e.g., g00gle).
- **Punycode:** Encoding scheme used to convert Unicode characters into ASCII format for URLs (often seen as `xn-`).